LOGISTICS · FIELD OPERATIONS · PRODUCT SPECIFICATION

ShiftPilot

Real-Time Dispatch & Field Operations Platform

Implementation-Ready Product Specification

Prepared for: Obieda Al-Darrabish — Portfolio Engineering Program

1. Project Overview

Project name: ShiftPilot **Tagline:** Every task, tracked. Every agent, on time. **Short description:** ShiftPilot is a real-time dispatch and field-operations platform that assigns, tracks, and analyzes tasks for delivery drivers, service technicians, and field-sales agents — replacing WhatsApp-and-spreadsheet coordination with a live, auditable system.

Full description: ShiftPilot gives operations teams a single place to create field tasks (a delivery, a home-service visit, a sales call), assign them to available agents (manually or via a rule-based auto-assign engine), and track them through a real-time status pipeline from creation to completion. Agents interact through a lightweight mobile-friendly view showing their assigned tasks and one-tap status updates. Every status change streams live to the dispatcher's dashboard over WebSockets, triggers webhook notifications to external systems (e.g., a customer-facing SMS), and feeds a performance-analytics layer (on-time rate, tasks per agent, idle time). A scheduled SLA-monitor flags tasks at risk of breaching their time window before they actually do.

Main objective: Replace ad-hoc, invisible field-coordination workflows with a structured, real-time, data-generating system — for companies that currently run field operations "by phone call."

Why this product should exist: Amman has a dense cluster of businesses that depend on field operations — last-mile delivery, home maintenance and cleaning services, home medical visits, and B2B field sales — and the overwhelming majority coordinate these operations manually. This is exactly the gap that Jordanian field-sales SaaS company Repzo was built to address, evidence that the market genuinely feels this pain and will pay to solve it.

What real-world problem it solves: Operations managers have no live visibility into where their field agents are in their task list, no historical data to evaluate agent performance, and no systematic way to flag a task that's about to breach a promised time window until the customer complains. ShiftPilot turns invisible, undocumented coordination into a structured, queryable, real-time system of record.

Who suffers from this problem: Dispatchers, who currently juggle phone calls and group chats to figure out who's free; operations managers, who have no data to improve routing or staffing decisions; customers, who get vague or wrong ETAs; and the field agents themselves, who receive conflicting or late instructions.

Why current solutions are insufficient: Spreadsheets and messaging apps have no state machine (nothing stops two dispatchers from double-assigning the same task), no audit trail, no real-time visibility, and produce zero usable performance data. Enterprise fleet-management software exists but is typically expensive, over-featured, and built for large logistics operations rather than the SME segment.

Why this project is valuable in Jordan / Amman: It targets a genuinely underserved segment (SME field operations) in a market where the closest comparable product (Repzo) validates real demand, and it demonstrates exactly the real-time, event-driven, microservice-adjacent backend skills that Amman's Java/Spring Boot shops (ICSFS, ESKADENIA) and product companies look for.

Why it is valuable as a software-engineering portfolio project: It proves the candidate can design and build a real-time system — not just request/response CRUD — including WebSocket architecture, pub/sub messaging, background scheduling, and a genuine state machine with concurrency safety (preventing double-assignment races). This is a materially different, more advanced skill signal than a typical junior project.

---

2. Target Users

User Type 1: Dispatcher ("Lina, 29, operations coordinator at a home-cleaning service")

- **Who they are:** The person who creates tasks and assigns them to field agents throughout the day.
- **What they need:** A live, at-a-glance view of every agent's status and workload, and a fast way to assign new tasks as they come in.
- **Pain points:** Currently juggles 5 WhatsApp group chats and a shared spreadsheet; loses track of who's already been assigned three jobs today.
- **What they can do inside the system:** Create/edit/cancel tasks, assign tasks manually or trigger auto-assign, view the live agent map/status board, message an agent (future feature), view SLA-risk alerts, run daily reports.
- **What they cannot do:** Access billing/company-plan settings (Admin only), permanently delete historical task records (compliance/audit requirement — cancel, not delete).
- **Why they would use the product:** It replaces five browser/app tabs and constant phone calls with one live board.

User Type 2: Field Agent ("Omar, 24, delivery driver")

- **Who they are:** The person physically completing tasks — driving, visiting, servicing.
- **What they need:** A dead-simple mobile view: "here's my next task, here's the address, here's the one button I press when I'm done."
- **Pain points:** Confusing multi-step apps eat into paid work time; unreliable connectivity in the field.
- **What they can do:** View assigned tasks in priority order, accept/decline a task, update status (en route → arrived → completed), see basic task details (address, customer name, notes), view their own daily completion count.
- **What they cannot do:** See other agents' tasks or performance data; reassign tasks to themselves; edit task details set by the dispatcher.
- **Why they would use the product:** Fewer confused phone calls asking "where do I go next," and a clear record protecting them if a customer disputes a visit.

User Type 3: Operations Manager ("Sami, 38, owns the cleaning-service business")

- **Who they are:** The business owner/manager who cares about overall performance, not day-to-day dispatch.
- **What they need:** Analytics — on-time rate trends, agent performance comparison, busiest hours/zones — to make staffing and pricing decisions.
- **What they can do:** View all dispatcher-level data plus analytics dashboards, manage agent roster, configure SLA windows and auto-assign rules, manage company settings and billing.
- **What they cannot do:** Nothing within their own company's data — they have full administrative access; they cannot see other companies' data (tenant isolation).
- **Why they would use the product:** Turns "I think Fridays are busy" into an actual chart he can act on.

User Type 4: Customer (indirect user, via notification only)

- **Who they are:** The end customer receiving the service.
- **What they need:** To know roughly when the agent will arrive.
- **What they can do:** Receive a status notification (SMS/webhook-triggered) when their task is assigned and when the agent is en route — no login, no account, purely a notification recipient in the MVP.
- **Why relevant:** The whole system's value ultimately shows up as a better customer experience, even though the customer never logs into ShiftPilot directly.

---

3. Complete Product Workflow

3.1 First-Time User Journey (Operations Manager)

1. Sami signs up, creates a company account, verifies email.
2. **Onboarding Step 1:** Company profile — name, service type (delivery/home-service/field-sales), timezone.
3. **Onboarding Step 2:** Add agents — invite by email/phone, or bulk-import a CSV of agent names/phones.
4. **Onboarding Step 3:** Configure a default SLA window (e.g., "tasks should be completed within 2 hours of creation") and choose manual vs. auto-assign as the default.
5. Lands on the **Dispatch Board**, sees a seeded sample task move through the pipeline live (a guided "watch it work" demo task).

3.2 Registration / Login

- Standard email/password registration with company creation; agents are invited (not self-registering) by a dispatcher/manager, receiving an SMS/email invite link to set a password and install the mobile-friendly web app.
- Login: email/password (dispatchers/managers) or phone/PIN (agents, optimized for quick field access) → JWT access token + refresh token.

3.3 Core Workflow — Creating and Completing a Task

```
Dispatcher creates task (customer info, address, time window, task type)
  → Backend validates payload
  → Task saved with status = UNASSIGNED
  → Auto-assign engine evaluates (if enabled):
      - filters agents who are ON_SHIFT and not over capacity
      - scores by proximity (if location available) and current load
      - assigns to best-scoring agent, OR leaves UNASSIGNED for manual pick if no good match
  → Assigned agent's app receives a real-time push (WebSocket) + notification
  → Agent taps "Accept" → status = ACCEPTED
  → Agent taps "En Route" → status = EN_ROUTE → dispatcher board updates live
  → Agent taps "Arrived" → status = ARRIVED
  → Agent taps "Completed" (+ optional photo/notes) → status = COMPLETED
  → System records completion timestamp, computes actual-vs-SLA duration
  → Webhook fired: task.completed → external system notified (e.g., triggers customer SMS)
  → Analytics aggregates update (on-time yes/no, duration, agent tally)
```

3.4 SLA-Risk Workflow (Background)

1. A scheduled job runs every minute, scanning all `UNASSIGNED`, `ACCEPTED`, and `EN_ROUTE` tasks.
2. For each task, it compares `now` against `sla_deadline`.
3. If within a configurable risk window (e.g., 20 minutes before breach) and not yet flagged, the task is marked `at_risk = true`, a dashboard alert fires, and (optionally) a webhook `task.at_risk` is sent.
4. If the deadline passes with the task not `COMPLETED`, status becomes `BREACHED` (task continues, just flagged), feeding into the manager's performance analytics as a missed SLA.

3.5 Admin / Roster Workflow

1. Manager goes to **Team → Agents** → **Add Agent** → enters name, phone, shift schedule.
2. Backend creates an `Agent` record in `Invited` state, sends an SMS/email invite.
3. Agent sets a PIN via the invite link → status becomes `Active`.
4. Manager can toggle an agent `On Shift`/`Off Shift`, which the auto-assign engine respects.

3.6 Notifications

- In-app/WebSocket push to agents on new assignment.
- In-app toast to dispatchers on SLA-risk alerts and task completion.
- Webhook events fired to the company's configured endpoint for `task.created`, `task.assigned`, `task.status_changed`, `task.completed`, `task.at_risk`, `task.breached`.
- (Optional/future) Customer-facing SMS via a stubbed integration point, triggered from the `task.assigned`/`task.en_route` webhook.

3.7 Success / Error States

- **Success:** task pipeline advances cleanly through each stage, dashboard shows a green "On Time" tag on completion.
- **Error — no agent available for auto-assign:** task stays `UNASSIGNED`, surfaced prominently in a "Needs Manual Assignment" list rather than silently failing.
- **Error — agent declines a task:** returns to `UNASSIGNED`, auto-assign (if enabled) tries the next-best agent automatically.
- **Error — WebSocket disconnect (agent has poor signal):** client falls back to polling every 30s until reconnected; status updates queue locally and sync on reconnect (no data loss, just delayed visibility).

---

4. Complete Feature List

MVP

Feature	What it does	Why it exists	Who uses it	Behavior	On failure	Dependencies
Task CRUD	Create/edit/cancel field tasks	Core unit of work	Dispatcher	Standard REST	Validation errors returned inline	Auth
Agent roster management	Add/invite/manage field agents	Need a pool to assign to	Manager	Invite flow, shift toggle	Duplicate-phone error	Auth
Manual assignment	Dispatcher picks an agent for a task	Baseline assignment method	Dispatcher	One click, respects on-shift status	Blocks assigning to off-shift agent	Task, Agent
Status state machine	Enforces valid task-status transitions	Data integrity	Agent (via app), system	UNASSIGNED→ACCEPTED→EN_ROUTE→ARRIVED→COMPLETED, or →DECLINED/CANCELLED	Rejects invalid transitions (e.g., ARRIVED→UNASSIGNED)	—
Real-time status feed	Live dashboard updates via WebSocket	Removes need to refresh/call	Dispatcher	Push on every status change	Falls back to polling	WebSocket infra
Basic analytics	Completion counts, on-time %	Baseline visibility	Manager	Daily aggregation	Cached last-known value shown if job delayed	Task history
Auth + roles	Secure multi-tenant, multi-role access	Baseline requirement	All	JWT + RBAC	401/403	—

Version 1.1

Feature	What it does	Why it exists	Who uses it
Rule-based auto-assignment	Assigns automatically by proximity/load	Removes manual dispatch bottleneck	System, Dispatcher (configures rules)
SLA-risk alerts	Flags at-risk tasks before breach	Proactive rather than reactive ops	Dispatcher
Webhook notifications	Pushes status events externally	Integrates with existing tools	Developer/Manager
Agent performance leaderboard	Ranks agents by on-time rate/volume	Motivation + staffing decisions	Manager

Version 2

Feature	What it does	Why it exists
Live map view with agent locations	Visual dispatch, real routing awareness	Larger fleets need spatial context
Route ordering / multi-stop optimization	Suggests an efficient order for an agent's tasks	Efficiency at scale
Customer-facing tracking link	Live ETA page for the end customer	Customer-experience differentiator
Multi-branch / multi-region support	Separate boards per branch, rolled up centrally	Larger companies

Advanced / Enterprise

Feature	What it does	Why it exists
Full route optimization engine	Multi-constraint solver (time windows, vehicle capacity)	Serious logistics operations
Offline-first agent app (PWA sync)	Works with zero connectivity, syncs later	Rural/low-signal areas
SSO / enterprise auth	IT requirement for larger customers	Enterprise sales

Optional Future Ideas

- In-app chat between dispatcher and agent.
- Photo/signature proof-of-completion.
- Predictive staffing suggestions based on historical demand patterns.

5. UI / UX Design Specification

Design style: High-energy, high-clarity operations-console aesthetic — think air-traffic-control-board meets modern SaaS. Bold color-coded status states so a dispatcher can read the entire board's health at a glance without reading a single word.

Brand personality: Fast, confident, kinetic, dependable under pressure — the opposite of ClearVoice's quiet institutional calm. ShiftPilot should feel like a live cockpit, not a filing cabinet.

Overall visual direction: A dark charcoal-navy console shell (reduces eye strain for all-day dispatcher use, and reads as "professional ops tool" rather than "consumer app") with a vivid orange as the primary action/brand color and teal as the secondary/informational accent — a deliberately different palette family from ClearVoice's navy/gold to signal a distinct product.

Color Palette (exact HEX)

Role	HEX	Usage
Primary (console shell)	`#1F2A44`	Sidebar, top bar, headers
Secondary / Brand action	`#E85D2A`	Primary buttons, active nav item, key highlights
Accent	`#17B0A7`	Secondary actions, info badges, links
Background	`#F4F5F7`	Main content area (light mode for dashboard body)
Surface / card	`#FFFFFF`	Cards, panels
Status — Unassigned	`#8B93A7`	Gray badge
Status — Accepted/En Route	`#2E86DE`	Blue badge
Status — Arrived	`#17B0A7`	Teal badge
Status — Completed / On Time	`#1E8E5A`	Green badge
Status — At Risk	`#E8A62A`	Amber badge
Status — Breached / Cancelled	`#D64545`	Red badge
Text — primary	`#161B29`	Body text
Text — secondary	`#5B6474`	Captions, metadata
Text — on-dark	`#F4F5F7`	Text on the console shell
Border	`#E3E6EA`	Card/table dividers

Typography

- **Recommended fonts:** Heading — *Sora* or *Space Grotesk* (geometric, technical, energetic — fits an "ops console" feel). Body/UI — *Inter*. Monospace (task IDs, timestamps in logs) — *JetBrains Mono*.
- **Heading sizes:** H1 26px, H2 19px, H3 15px — slightly tighter than ClearVoice to fit more live data on screen.
- **Body sizes:** Base 14px, dense-table text 12.5px (the dispatch board prioritizes information density).
- **Font weights:** Headings 700 (bold, confident); body 400; status badges 600 (semibold, easy to scan).
- **Line spacing:** 1.4 body, 1.2 headings, 1.3 table rows (tighter than ClearVoice — this is a live-monitoring tool, not a document-reading tool).

Spacing System

- **Base unit:** 4px grid.
- **Page margins:** 24px desktop (tighter than ClearVoice to maximize board real estate), 12px mobile.
- **Card padding:** 16px.
- **Card-to-card gap:** 12px.
- **Border radius:** 6px (slightly sharper/more technical than ClearVoice's 8px).
- **Button sizes:** Default 36px, compact 28px (agent mobile view uses large 52px touch targets instead — see §7).

Components

- **Buttons:** Primary — solid orange `#E85D2A`, white text, 6px radius. Secondary — outlined teal. On the agent mobile view, buttons are oversized (min 52px height) single-purpose action buttons ("I'm En Route," "Mark Arrived," "Complete Task") — one thumb, no menus.
- **Inputs:** Dark-shell-aware — white background with `#E3E6EA` border in the main app; auto-adapts to a higher-contrast style within the dark console chrome.
- **Dropdowns:** Compact, used for status filters/agent filters on the board.
- **Tables:** The **Dispatch Board** is the signature component — not a plain table but a Kanban-style multi-column live board (Unassigned / Accepted / En Route / Arrived / Completed), each column a scrollable list of task cards, cards color-left-bordered by status/SLA-risk. A secondary flat table view exists for the Task History screen.
- **Cards (task cards on the board):** Compact, white, 6px radius, left border 4px colored by status, showing: task ref, customer name, address (truncated), assigned agent avatar/initials, SLA countdown chip (turns amber then red as deadline approaches).
- **Modals:** Task detail expands as a right-side slide-over panel (not a centered modal) — keeps the board visible behind it, matching an "ops console" pattern rather than an interrupting dialog.
- **Toast notifications:** Bottom-right, colored left border, brief and dense (dispatchers see many of these per hour, so they must not be intrusive).
- **Tabs:** Pill-style tabs (not underline) for switching between Board / Map / Analytics / History views — a bolder, more app-like feel than ClearVoice's understated underline tabs.
- **Navigation:** Fixed dark sidebar (charcoal-navy `#1F2A44`), orange active-state left bar, icon + label.

- **Sidebar:** 220px, collapsible to 60px icon rail.
- **Search:** Board-level quick search/filter by task ref, customer name, or agent name.
- **Filters:** Persistent filter bar above the board (agent, status, SLA-risk toggle, date), shown as toggle chips.
- **Pagination:** Used only on the flat Task History table; the live board itself uses infinite-scroll columns instead of pagination (live data shouldn't force a page-click to see more).
- **Charts:** Bar chart for tasks-per-agent, line chart for on-time-rate trend, heat-map-style grid for busiest hours/day-of-week — all rendered in the primary/secondary/status palette, high-contrast against the light dashboard background.

---

6. Complete Screen List

6.1 Landing Page

- **Purpose:** Convert operations-heavy SMEs.
- **Layout:** Dark hero with a live-looking animated mock dispatch board (static screenshot with a subtle pulse animation on one card to imply "live"), orange CTA "Start Dispatching," feature strip below, footer.

6.2 Login (Dispatcher/Manager)

- **Purpose:** Authenticate desktop users.
- **Layout:** Centered card on the dark console background, email/password, orange primary button.

6.3 Agent Login (Mobile)

- **Purpose:** Fast field login.
- **Layout:** Full-screen mobile form, phone number + 4-digit PIN (numeric keypad UI), single large "Log In" button — deliberately simpler than the desktop login to minimize friction for field workers.

6.4 Onboarding Wizard

- **Purpose:** First-run company + agent setup.
- **Layout:** 3-step wizard matching the journey in §3.1, orange progress indicator.

6.5 Dispatch Board (Home — Dispatcher/Manager)

- **Purpose:** The core live-operations screen.
- **Layout:** Kanban-style columns (Unassigned, Accepted, En Route, Arrived, Completed) each independently scrollable, task cards as described in §5, persistent filter bar above, "+ New Task" orange button top-right, live WebSocket connection-status indicator (small green/red dot) in the corner.
- **Empty state:** "No active tasks — create one to get started" with a "+ New Task" CTA.
- **Loading state:** Skeleton cards per column.
- **Error state (WebSocket disconnected):** A thin amber banner: "Live updates paused — reconnecting..." with the board continuing to show last-known data.

6.6 New/Edit Task Modal

- **Purpose:** Create or edit a task.
- **Layout:** Right-side slide-over panel: customer name, phone, address (with map-pin picker in v2), task type, time window, notes, "Assign now" toggle (manual picker vs. auto-assign).

6.7 Task Detail (Slide-over)

- **Purpose:** Full task lifecycle view.
- **Layout:** Header (task ref, status badge, SLA countdown), timeline of status changes with timestamps, assigned agent card, action buttons (Reassign, Cancel), notes/photo section (v1.1+).

6.8 Live Map View (v2)

- **Purpose:** Spatial dispatch awareness.
- **Layout:** Full-width map, agent pins color-coded by status, task pins, click-to-assign-nearest interaction.

6.9 Agent Task List (Mobile — Agent view)

- **Purpose:** The agent's entire app experience.
- **Layout:** Single-column list of assigned tasks in priority order, each a large tappable card (customer name, address, time window, one prominent action button matching current status: Accept / En Route / Arrived / Complete), pull-to-refresh.
- **Empty state:** "No tasks assigned right now — check back soon."
- **Loading state:** Simple spinner (agents' phones may be lower-spec; keep this screen extremely light).

6.10 Task History (Flat table)

- **Purpose:** Searchable historical record for reporting/audit.
- **Layout:** Standard filterable/paginated table (unlike the live board), export-to-CSV button.

6.11 Analytics Dashboard

- **Purpose:** Manager-level performance insight.
- **Layout:** Stat cards (on-time %, avg completion time, total tasks today), bar chart (tasks per agent), line chart (on-time trend, 30 days), heat-map grid (busiest hours).

6.12 Team / Agents

- **Purpose:** Roster management.

- **Layout:** Table of agents (name, phone, shift status toggle, tasks-today count), "Add Agent" button.

6.13 Settings — Company & SLA Rules

- **Purpose:** Configure SLA windows, auto-assign rules, webhook endpoints.
- **Layout:** Tabbed settings sections: Company Profile, SLA & Auto-Assign, Webhooks, Notifications.

6.14 Error Pages (404/500/403)

- **Purpose:** Graceful failure, consistent with the dark console brand.

---

7. Responsive Design

- **Desktop (≥1280px):** Full Kanban board with all 5 columns visible side-by-side; sidebar expanded.
- **Laptop (1024–1279px):** Board columns become horizontally scrollable (snap-scroll) rather than shrinking illegibly; sidebar remains.
- **Tablet (768–1023px):** Sidebar collapses to icon rail; board shows 2–3 columns at a time with swipe/scroll; task detail becomes a full-screen overlay instead of a slide-over.
- **Mobile — Dispatcher/Manager view (≤767px):** The Kanban board is deliberately **not** used on mobile for the dispatcher role (a 5-column board doesn't work on a phone) — it degrades to a single filterable list view, status shown as a colored left-border on stacked cards, tapping opens task detail full-screen.
- **Mobile — Agent view:** This is the *primary* form factor for agents (§6.9 is mobile-first by design, not a responsive fallback) — large touch targets (min 52px), single-column, minimal data per screen, works acceptably on 3G.
- **Navigation changes:** Desktop/tablet sidebar → mobile bottom tab bar (Board/List, History, Analytics, Settings) for dispatcher/manager; agents get no sidebar at all — just their task list and a small profile icon.
- **Touch interactions:** Swipe-to-advance-status on agent task cards (swipe right = advance to next status) as a faster alternative to tapping the button, with the button always available as the accessible/explicit fallback.

8. Frontend Architecture

- **Framework:** React 18 + TypeScript for the dispatcher/manager web app; the agent view is the **same React codebase** rendered as a mobile-optimized route set (not a separate native app) to keep the MVP scope realistic, structured so it could be wrapped as a PWA later.
- **State management: TanStack Query** for server state (tasks, agents, analytics), with a **WebSocket-driven cache-patching layer**: incoming real-time events directly update the TanStack Query cache (via `queryClient.setQueryData`) rather than triggering a full refetch — this is the architecturally interesting part of the frontend and a strong interview talking point. **Zustand** for local UI state (selected filters, board scroll position).
- **API layer:** Typed REST client generated from OpenAPI, plus a dedicated `useTaskSocket()` hook wrapping a native WebSocket connection with auto-reconnect and exponential backoff.
- **Authentication handling:** JWT access token in memory; refresh token in httpOnly cookie for the desktop app; for the agent mobile view, a longer-lived token strategy is used (field agents shouldn't be logged out mid-shift due to token expiry), balanced against security via device-bound refresh tokens.
- **Form handling:** React Hook Form for task-creation/settings forms.
- **Validation:** Zod schemas.
- **Routing:** React Router v6 with two top-level layouts: `ConsoleLayout` (sidebar, dispatcher/manager) and `FieldLayout` (bottom-nav-free, agent, minimal chrome).

Suggested Frontend Folder Structure

```
frontend/
  src/
    app/
    features/
      board/            # Kanban board, task cards, slide-over
      tasks/             # Task CRUD forms
      agents/            # Roster management
      analytics/
      field/             # Agent-facing mobile views
      settings/
    components/
      ui/
      board/             # BoardColumn, TaskCard, SlaCountdown
      layout/            # ConsoleLayout, FieldLayout
    lib/
      api-client/
      websocket/          # useTaskSocket, reconnect logic
      auth/
      hooks/
    styles/
      theme.css
    types/
```

9. Backend Architecture

- **Backend language:** Java 17.
- **Framework:** Spring Boot 3 (Spring Web, Spring WebSocket/STOMP, Spring Data JPA, Spring Security).
- **Architecture pattern:** Layered/hexagonal — Controller → Service → Repository, with a clear domain model (`Task`, `Agent`, `Assignment`) independent of persistence concerns, and a dedicated `realtime` module for WebSocket/pub-sub concerns kept separate from core business logic.
- **Layers:**
- **API layer** — REST controllers + a WebSocket/STOMP endpoint for the live board feed.
- **Service layer** — `TaskService`, `AssignmentService` (encapsulates the state machine + auto-assign logic), `SlaMonitorService`, `AnalyticsService`.
- **Domain layer** — `Task` entity with an enforced status state machine (illegal transitions throw a domain exception, not a generic 500), `Agent`, `Assignment`.
- **Data-access layer** — Spring Data JPA repositories over PostgreSQL.
- **Integration layer** — `WebhookDispatchService`, (future) SMS provider client.
- **Services:** `TaskService` (CRUD + transitions), `AutoAssignService` (scoring/matching engine), `RealtimeBroadcastService` (publishes to Redis pub/sub → WebSocket sessions), `SlaMonitorService` (scheduled scan), `AnalyticsService` (aggregation queries), `WebhookDispatchService`.
- **Controllers:** `TaskController`, `AgentController`, `AuthController`, `AnalyticsController`, `WebhookController`, plus a `TaskSocketController` (`@MessageMapping` STOMP handlers) for real-time.
- **Domain layer:** The `Task.transitionTo(newStatus)` method is where the state machine lives — it is the single source of truth for which transitions are legal, so the rule can never be duplicated or drifted between the API layer and a background job.
- **Data-access layer:** Spring Data JPA + PostgreSQL, using optimistic locking (`@Version` column on `Task`) to prevent the classic double-assignment race condition (two dispatchers assigning the same unassigned task simultaneously) without heavy pessimistic locking.
- **Integration layer:** Webhook dispatch decoupled from the request path via Spring's `@Async` + a retry-capable delivery mechanism.
- **Workers:** A `@Scheduled` SLA-monitor job (runs every 60s); a webhook-delivery worker consuming from a lightweight queue table (outbox pattern, same approach as ClearVoice, applied here in Java/Spring form).
- **Queues:** Redis is used for two distinct purposes — (1) **pub/sub** for real-time broadcast (a status change publishes to a Redis channel, which every API instance subscribes to and forwards to its locally-connected WebSocket clients, enabling horizontal scaling of the API layer), and (2) as the backing store for the webhook-delivery outbox (via a Redis list/stream, or a Postgres table for simplicity in MVP — documented as a deliberate choice, see §15).
- **Schedulers:** Spring's `@Scheduled` annotation for the SLA monitor and daily analytics rollup job.

Component communication: REST controllers call Services → Services mutate Domain entities (which self-enforce valid transitions) → Repository persists via JPA → on a successful transition, the Service publishes a domain event → a listener broadcasts it to Redis pub/sub → every API instance's WebSocket layer receives the pub/sub message and pushes it to its connected clients whose subscriptions match (dispatcher viewing that company's board) → separately, the same domain event triggers the webhook-delivery worker.

---

10. Complete System Architecture



Both the dispatcher web app and the agent mobile view connect to the same stateless Spring Boot API over REST for CRUD operations and over WebSocket (STOMP over SockJS) for real-time updates. Redis pub/sub is the mechanism that lets a status change originating on one API instance (handling agent Omar's "Arrived" tap) reach a dispatcher's browser connected to a *different* API instance — this is what makes the real-time layer horizontally scalable rather than tying clients to a single server. PostgreSQL is the system of record; Redis is deliberately treated as ephemeral (losing it degrades real-time speed to polling, but never loses data, since PostgreSQL remains the source of truth).

11. Database Design

`companies`

Field	Type	Constraints
id	uuid	PK
name	varchar(255)	required
service_type	varchar(50)	required (`delivery`, `home_service`, `field_sales`)
timezone	varchar(50)	default 'Asia/Amman'
default_sla_minutes	int	default 120
created_at	timestamptz	required

`users` (dispatchers/managers)

Field	Type	Constraints
id	uuid	PK
company_id	uuid	FK → companies.id
email	varchar(255)	required, unique
password_hash	varchar(255)	required
role	varchar(20)	required (`Manager`, `Dispatcher`)
created_at	timestamptz	required

`agents`

Field	Type	Constraints
id	uuid	PK
company_id	uuid	FK → companies.id, indexed
full_name	varchar(255)	required
phone	varchar(30)	required, unique per company
pin_hash	varchar(255)	required
status	varchar(20)	default 'invited' (`invited`, `active`, `deactivated`)
on_shift	boolean	default false
current_load	int	default 0 (denormalized count of active tasks, updated transactionally)
created_at	timestamptz	required

`tasks`

Field	Type	Constraints
id	uuid	PK
company_id	uuid	FK → companies.id, indexed
reference_code	varchar(20)	required, unique per company (human-friendly short ID)
customer_name	varchar(255)	required
customer_phone	varchar(30)	nullable
address	text	required
task_type	varchar(50)	required
notes	text	nullable
status	varchar(20)	required (`UNASSIGNED`, `ACCEPTED`, `EN_ROUTE`, `ARRIVED`, `COMPLETED`, `DECLINED`, `CANCELLED`)
sla_deadline	timestamptz	required
at_risk	boolean	default false
breached	boolean	default false
version	int	required (optimistic locking column)
created_at	timestamptz	required
completed_at	timestamptz	nullable
index		(company_id, status), (company_id, sla_deadline)

`assignments`

Field	Type	Constraints
id	uuid	PK
task_id	uuid	FK → tasks.id, required, unique (one active assignment per task)
agent_id	uuid	FK → agents.id, required
assigned_at	timestamptz	required
assignment_method	varchar(20)	required (`manual`, `auto`)
unassigned_at	timestamptz	nullable (set if reassigned/declined)

`status_events`

Field	Type	Constraints
id	uuid	PK
task_id	uuid	FK → tasks.id, indexed
from_status	varchar(20)	nullable
to_status	varchar(20)	required
changed_by	uuid	nullable (agent or user id; null = system)
occurred_at	timestamptz	required

`webhook_endpoints` / **`webhook_deliveries`**

(Same shape/purpose as ClearVoice §11 — company-scoped URL + secret, delivery log with status/attempt count.)

`outbox_messages`

(Same pattern as ClearVoice — used here specifically for webhook delivery, since real-time broadcast uses Redis pub/sub directly rather than the outbox.)

Relationships

- `companies` 1-to-many `users`, `agents`, `tasks`, `webhook_endpoints`.
- `tasks` 1-to-many `status_events`; `tasks` 1-to-one active `assignments` (enforced via unique constraint on `task_id` where `unassigned_at IS NULL`).
- `agents` 1-to-many `assignments` (historical).
- No many-to-many relationships in MVP.

---

12. API Design

Base path `/api/v1`. Dispatcher/manager auth via JWT bearer; agent auth via a device-scoped JWT issued from phone+PIN login. Consistent envelope as ClearVoice (`data`/`error`).

Auth Module

Method	URL	Purpose	Auth
POST	/auth/register	Create company + manager account	None
POST	/auth/login	Dispatcher/manager login	None
POST	/auth/agent-login	Agent phone+PIN login	None
POST	/auth/refresh	Rotate token	Refresh cookie/token
POST	/auth/logout	Revoke session	Auth

Tasks Module

Method	URL	Purpose	Auth	Request	Response	Success	Errors
POST	/tasks	Create a task	JWT (Dispatcher+)	customer/address/type/SLA	Task object	201	400
GET	/tasks	List/filter tasks	JWT	query: status, agentId, date	Paginated list	200	400
GET	/tasks/{id}	Task detail	JWT	—	Task + status history	200	404
PATCH	/tasks/{id}	Edit task details	JWT (Dispatcher+)	partial fields	Updated task	200	404, 409 (already completed)
POST	/tasks/{id}/assign	Manually assign to an agent	JWT (Dispatcher+)	`{agentId}`	Assignment	200	409 (already assigned — optimistic lock conflict)
POST	/tasks/{id}/auto-assign	Trigger auto-assign engine	JWT (Dispatcher+)	—	Assignment or "no match"	200	422 (no eligible agent)
POST	/tasks/{id}/status	Agent updates status	JWT (Agent)	`{status}`	Updated task	200	409 (illegal transition)
POST	/tasks/{id}/cancel	Cancel a task	JWT (Dispatcher+)	`{reason}`	Updated task	200	409 (already completed)

Agents Module

Method	URL	Purpose	Auth
GET	/agents	List agents	JWT
POST	/agents	Invite new agent	JWT (Manager)
PATCH	/agents/{id}	Update shift status/details	JWT (Manager or self)
GET	/agents/{id}/tasks	Agent's own assigned tasks	JWT (Agent, self-only)

Analytics Module

Method	URL	Purpose	Auth
GET	/analytics/summary	On-time %, totals	JWT (Manager)
GET	/analytics/agents	Per-agent performance	JWT (Manager)
GET	/analytics/trend	Daily time series	JWT (Manager)

Webhooks / System

Method	URL	Purpose	Auth
GET/POST/DELETE	/webhooks	Manage endpoints	JWT (Manager)
GET	/health, /health/ready	Health checks	None

Real-Time (WebSocket/STOMP)

- Connect: `wss://.../ws` (SockJS fallback for restrictive networks).
- Subscribe: `/topic/company/{companyId}/board` (dispatcher board updates) and `/topic/agent/{agentId}` (agent's own task push).
- Server → client message: `{ eventType: "task.status_changed", task: {...} }`.

13. Authentication & Authorization

- **Registration:** Manager registers with email/password, creating the `Company` and their own `User` record; password hashed with **BCrypt** (Spring Security's default, well-supported in the Java ecosystem — a deliberate, explainable choice, contrasted against Argon2id in the ClearVoice document as an interview talking point about trade-offs).
- **Login (Manager/Dispatcher):** Standard email/password → JWT access token (15 min) + refresh token (7 days, httpOnly cookie).
- **Login (Agent):** Phone number + 4-digit PIN, optimized for fast field entry; PIN hashed identically (BCrypt); agent tokens are longer-lived (e.g., 12-hour access token) since re-authenticating mid-shift is a real operational annoyance, offset by device binding (token includes a device fingerprint claim, and a manager can force-revoke a specific agent's session from the roster screen).
- **Password/PIN hashing:** BCrypt, cost factor 12.
- **JWT/session strategy:** Same stateless-access + stateful-refresh pattern as ClearVoice; agent tokens carry an additional `deviceId` claim.
- **Refresh tokens:** Rotated on use; a manager-triggered "revoke agent session" immediately invalidates the stored refresh token, effectively remote-logging-out a lost/stolen phone.
- **Password reset:** Manager/dispatcher self-service via emailed link; agent PIN reset is manager-initiated (agents call the office, not "forgot PIN" self-service, matching realistic field-ops practice).
- **Email verification:** Required for manager/dispatcher accounts; not applicable to agents (invited via a trusted channel by the manager).
- **Roles:** `Manager` (full company access + billing/settings), `Dispatcher` (task/agent operational access, no billing/settings), `Agent` (self-scoped task access only).
- **Permissions:** Spring Security method-level `@PreAuthorize` annotations mapped to a role-permission matrix; agents are additionally restricted to *their own* assigned tasks via an ownership check in the service layer (`assignment.agentId == principal.agentId`), not just a role check.
- **Admin privileges:** `Manager` can manage the agent roster, SLA rules, and webhooks; `Dispatcher` cannot change SLA rules or webhook configuration (prevents accidental operational-policy changes by day-to-day staff).
- **Tenant isolation:** Every query filtered by `company_id` derived from the JWT, enforced via a Spring Data JPA specification/filter applied globally, identical philosophy to ClearVoice's approach.

14. Security

- **OWASP considerations:** Same Top-10-driven checklist approach as ClearVoice, with two ShiftPilot-specific emphases: **Broken Access Control** around the agent "own tasks only" boundary (the single most likely mistake in a multi-role field-ops app — a bug here would let one agent see another's customer data), and **Insecure Design** around the optimistic-locking assignment race (§9/§11) which, if absent, would let two dispatchers double-book the same task.
- **Input validation:** Bean Validation (`jakarta.validation`) annotations on all request DTOs; task status transitions validated in the domain layer regardless of what the API layer already checked (defense in depth — never trust a single validation point).
- **SQL injection:** Spring Data JPA parameterized queries throughout; no native/raw SQL string concatenation.
- **XSS:** React's default escaping; task notes/customer names are plain text fields rendered as text, never as HTML.
- **CSRF:** SameSite=Strict refresh cookie; state-changing requests require a custom header.
- **Authentication attacks:** Agent PIN login is rate-limited more aggressively than manager email/password (4-digit PINs are brute-forceable in principle — mitigated by strict per-phone-number lockout after 5 failed attempts and a mandatory cooldown, plus device-fingerprint anomaly logging).
- **Rate limiting:** Per-company and per-agent rate limits on the status-update endpoint (prevents a buggy/malicious client from spamming status flaps).
- **Secrets management:** JWT signing key, DB credentials, Redis credentials, webhook secrets in environment variables locally / managed secret store in cloud.
- **Encryption:** TLS everywhere; customer phone numbers treated as PII with restricted field-level logging (never written to plain-text application logs).
- **HTTPS:** Enforced, HSTS.
- **API security:** WebSocket connections authenticate via the same JWT (passed during the STOMP CONNECT frame), and each subscription is authorized server-side (an agent cannot subscribe to another agent's topic, a dispatcher cannot subscribe to another company's board topic) — this is verified explicitly, not assumed from the connection alone.
- **Access control:** Enforced at controller (`@PreAuthorize`), service (ownership checks), and WebSocket-subscription layers.
- **Audit logs:** `status_events` table (§11) doubles as both operational history and an audit trail of every task-status change with `changed_by`.
- **File-upload security:** Applies to the v1.1+ "photo proof of completion" feature — size limits, MIME validation, images re-encoded (not served as originally uploaded) to strip EXIF/GPS metadata and neutralize any embedded exploit payloads.
- **Prompt injection:** N/A — ShiftPilot's MVP has no AI component.

15. Automation & Event-Driven Workflows

- **Webhooks:** Company-configured endpoint receives `task.created`, `task.assigned`, `task.status_changed`, `task.completed`, `task.at_risk`, `task.breached` events, HMAC-signed.
- **Queues:** Two distinct mechanisms, chosen deliberately for their different jobs: **Redis pub/sub** for the low-latency, best-effort, high-frequency real-time board broadcast (losing an occasional pub/sub message just means a slightly stale board until the next event or a manual refresh — acceptable), and the **PostgreSQL outbox pattern** for webhook delivery, where reliability matters more than latency (a missed webhook could mean a customer never gets notified, which is a real business problem, not just a stale UI).
- **Workers:** `SlaMonitorJob` (`@Scheduled`, every 60s), `WebhookDeliveryWorker` (polls the outbox), `AutoAssignEngine` (invoked synchronously on task creation if enabled, not a background job — assignment should feel instant to the dispatcher).
- **Retry logic:** Webhook delivery retries with backoff (1m, 5m, 30m, 3 attempts total — shorter than ClearVoice's because a stale delivery-notification webhook is less consequential than a failed legal invoice clearance, an explicit, explainable difference in retry philosophy between the two products).
- **Scheduled jobs:** SLA-risk scan (60s), daily analytics rollup (nightly), weekly agent-performance digest email to managers.
- **Event handling:** A task-status transition is the central domain event; it triggers, in order: (1) persistence of the `status_events` row, (2) Redis pub/sub broadcast, (3) outbox-queued webhook dispatch, (4) analytics counters update (either inline for MVP simplicity or via the nightly rollup for heavier aggregates).
- **Idempotency:** The optimistic-locking `version` column on `Task` is the core idempotency/concurrency-safety mechanism — a status update or assignment request carrying a stale version is rejected with a 409, forcing the client to refetch and retry rather than silently overwriting a newer state (this is the direct solution to the "two dispatchers assign the same task" race condition named in §1/§9).
- **Failure recovery:** If the auto-assign engine finds no eligible agent, the task is left `UNASSIGNED` and surfaced prominently rather than endlessly retrying a search that won't succeed until a human intervenes (e.g., toggles an agent on-shift).
- **Dead-letter handling:** Webhook deliveries exceeding retry attempts are marked `dead` in `webhook_deliveries`, visible in the Settings → Webhooks delivery log for manual inspection/resend.

16. AI Architecture

Note: Like ClearVoice, ShiftPilot's MVP intentionally contains no AI in its core path — dispatch correctness is a deterministic, safety-relevant concern (assigning the wrong agent to a task has real operational cost), and an LLM has no place in that decision. The scoring-based **auto-assign engine** (§3.3, §15) is deliberately implemented as transparent, explainable rule-based logic (proximity/load scoring), not a black-box model, specifically so a dispatcher can always understand *why* a given agent was chosen.

- **Where AI could add value (future, optional, clearly bounded):** A predictive staffing suggestion — "based on the last 8 Fridays, you'll likely need 2 more agents on shift between 4–7pm" — is a forecasting task suited to a simple statistical/ML model (even a basic moving-average or lightweight regression) more than an LLM.
- **Which task it performs:** Demand forecasting from historical task-volume data, surfaced as a suggestion on the Analytics dashboard, never auto-applied to the schedule.
- **Model/provider options:** A lightweight time-series model (e.g., a simple seasonal-decomposition or a small hosted forecasting API) rather than a general-purpose LLM — this is genuinely a numerical-forecasting problem, and using an LLM for it would be exactly the "AI because it's trendy" mistake this specification is explicitly designed to avoid.
- **Input:** Historical `tasks` volume, grouped by day-of-week/hour.
- **Output:** A suggested staffing count per future shift window, with a confidence indicator.
- **Human validation:** The manager reviews the suggestion and manually adjusts the shift roster — the system never auto-schedules agents.
- **Guardrails / hallucination mitigation:** Not applicable in the LLM sense (no generative text output); the relevant safeguard is simply never trusting the forecast blindly — it is presented as a suggestion with visible historical context (a small chart showing the data it's based on), not a black-box number.
- **Cost controls:** Recomputed on a daily schedule (not per-request), trivially cheap at this scale.

17. External Integrations

Integration	Purpose	API	Auth	Data sent	Data received	Failure behavior
Company webhook endpoints	Push task lifecycle events	REST (company-provided URL)	HMAC signature	Task status event payload	HTTP status	Retried per §15; dead-lettered after max attempts
SMS provider (stub/future)	Notify customers of assignment/en-route	REST (e.g., a regional SMS gateway)	API key	Phone number + templated message	Delivery status	Non-blocking; failure logged, does not affect task status
Transactional email	Manager digests, agent invites, password resets	REST (SendGrid/Postmark-class)	API key	Recipient, template	Delivery status	Queued retry
Maps/geocoding provider (v2, for the Live Map view)	Convert addresses to coordinates, render map	REST (e.g., Google Maps/Mapbox)	API key	Address string	Lat/long	Falls back to address-only display without map pin

18. Performance & Scalability

- **Expected load (portfolio scale):** Low hundreds of tasks/day across seeded demo companies; the interesting scalability story here is architectural (horizontal API scaling via Redis pub/sub), not raw volume.
- **Database optimization:** Composite indexes on `(company_id, status)` and `(company_id, sla_deadline)` for the two dominant query patterns (board rendering, SLA scanning).
- **Indexing:** Unique index on `(company_id, reference_code)`; unique partial index on `assignments.task_id WHERE unassigned_at IS NULL` to enforce "one active assignment per task" at the database level, not just application logic.
- **Caching:** Agent roster and company SLA settings cached in Redis with short TTL and explicit invalidation on update (these change rarely but are read on every task creation).
- **Connection pooling:** HikariCP (Spring Boot's default) tuned for the expected concurrent-request volume.
- **Async processing:** Webhook dispatch and analytics rollup are fully decoupled from the request path via Spring's `@Async`/scheduled jobs; the WebSocket broadcast path is intentionally kept as low-latency as possible (published to Redis synchronously within the transition, but the *fan-out to clients* is inherently async/non-blocking from the originating request's perspective).
- **Horizontal scaling:** This is the architecture's central scalability story — because real-time state lives in Redis pub/sub rather than in-process WebSocket session memory tied to one server, the Spring Boot API can run as N replicas behind a load balancer with sticky-session-free WebSocket support, and any replica can serve any client regardless of which replica handled the original status-changing request.
- **Load balancing:** A standard load balancer (cloud-provider managed, or nginx) distributes both REST and WebSocket (upgraded HTTP) traffic across replicas.
- **CDN:** Static frontend assets served via CDN.
- **MVP vs. at-scale:** MVP: single Postgres instance, single Redis instance, 1–2 API replicas (enough to *demonstrate* the horizontal-scaling claim in the demo/interview even if not "needed" at portfolio traffic). At true scale: Redis Cluster, Postgres read replicas for analytics, and the webhook outbox migrating to a real broker (Kafka/RabbitMQ) if delivery volume grows significantly.

19. Testing Strategy

- **Unit tests:** Domain logic — the `Task` state-machine transition rules (JUnit 5) are the single highest-value unit-test target; auto-assign scoring logic tested with a matrix of agent-load/proximity scenarios.
- **Integration tests:** Spring Boot `@SpringBootTest` + **Testcontainers** (PostgreSQL + Redis) verifying the full assign → status-update → webhook-outbox-write path.
- **API tests:** Contract tests against the OpenAPI spec; a focused test suite specifically for the optimistic-locking conflict scenario (two concurrent assignment requests for the same task — assert exactly one succeeds with 200 and the other receives 409).
- **Database tests:** Migration tests (Flyway/Liquibase migration applies cleanly); constraint tests for the partial-unique-index "one active assignment" rule.
- **Authentication tests:** Agent-can-only-see-own-tasks isolation test (this is the single most important authorization test in the app, analogous to ClearVoice's tenant-isolation test); PIN lockout behavior.
- **Security tests:** Dependency scanning in CI; WebSocket subscription-authorization test (assert a client cannot subscribe to another company's board topic even with a valid JWT for a *different* company).
- **End-to-end tests:** Playwright covering: manager creates task → auto-assign fires → agent (second browser context) sees it push in real time → agent advances status → manager's board updates live without a page refresh (this specific "two simultaneous browser sessions, real-time sync" test is the strongest possible demonstration of the WebSocket architecture actually working, and should be the flagship e2e test showcased to reviewers).
- **Load tests:** k6 script simulating N concurrent WebSocket connections plus concurrent task-status updates, verifying broadcast latency stays low and no message loss occurs under moderate concurrent load.
- **What to test first:** (1) state-machine transition legality, (2) optimistic-locking assignment race, (3) agent task-ownership authorization, (4) real-time broadcast correctness, (5) everything else.

20. Logging & Monitoring

- **Application logs:** Structured JSON via SLF4J/Logback, correlation ID (trace ID) propagated across the REST request and any resulting async webhook/broadcast work.
- **Error logs:** Centralized exception handling (`@ControllerAdvice`) mapping domain exceptions (e.g., `IllegalStatusTransitionException`) to clean 409 responses while logging full context server-side.
- **Audit logs:** `status_events` table serves this role for task history; a separate lightweight `security_events` log captures auth failures/lockouts.
- **Request tracing:** Micrometer Tracing (successor to Spring Cloud Sleuth) exporting to a tracing backend, showing the path from an agent's status-update REST call through to the Redis publish and webhook-outbox write.
- **Performance metrics:** Micrometer metrics (task-creation rate, WebSocket connection count, broadcast latency, SLA-breach rate) exported to Prometheus.
- **Health checks:** Spring Boot Actuator `/actuator/health` (liveness) and a custom readiness check verifying both PostgreSQL and Redis connectivity (a Redis outage should degrade real-time features gracefully, not fail the whole app's readiness — documented as a deliberate "degrade, don't crash" design choice).
- **Uptime monitoring:** External health-check ping, same pattern as ClearVoice.
- **Alerting:** Email/webhook alert if SLA-breach rate crosses a threshold or if the webhook dead-letter count grows — operationally meaningful alerts, not just infrastructure pings.

21. DevOps

- **Git strategy:** Trunk-based, short-lived feature branches, conventional commit messages.
- **Branch structure:** `main` protected, PR + CI required.
- **Pull requests:** Template requiring tests for any state-machine or authorization change specifically (called out because these are the highest-risk change categories in this codebase).
- **Environment variables:** `.env.example` covering DB, Redis, JWT signing key, webhook defaults.
- **Docker:** Multi-stage Dockerfile for the Spring Boot API (Maven/Gradle build stage → slim JRE runtime stage); separate frontend Dockerfile.
- **Docker Compose:** API + PostgreSQL + Redis + frontend + a mail-catcher, one-command local bootstrap with seed data (a demo company, agents, and a live-moving simulated task for the "watch it work" onboarding effect even in local dev).
- **CI/CD:** GitHub Actions — PR pipeline runs `mvn test`/`gradle test` + frontend lint/test + dependency scan; merge-to-main pipeline builds/pushes Docker images and deploys to staging.
- **GitHub Actions:** `ci.yml` and `deploy.yml`, mirroring the ClearVoice pattern for consistency across the portfolio.
- **Staging:** Auto-deployed on every merge to `main`; this is the environment used for the live demo.
- **Production:** Manually promoted from staging.
- **Rollback strategy:** SHA-tagged images; additive-only DB migrations preferred; Redis being ephemeral means a rollback never has to worry about "migrating" real-time state.

---

22. Cloud Deployment

Recommendation: AWS, chosen deliberately as a *different* cloud from ClearVoice's Azure recommendation — demonstrating comfort across both major providers is itself a portfolio signal, and AWS's ElastiCache (managed Redis) + RDS pairing is a natural fit for this specific Spring Boot + Postgres + Redis architecture.

Cheapest Portfolio Deployment

- **Application hosting:** AWS App Runner (simplest managed-container option) or a small EC2 instance running Docker Compose.
- **Database hosting:** Amazon RDS for PostgreSQL, `db.t4g.micro` (free-tier eligible burstable instance).
- **Cache/pub-sub:** Amazon ElastiCache for Redis, smallest node type — this is not optional/skippable the way it might be for a simpler CRUD app, since Redis pub/sub is core to the real-time architecture being demonstrated.
- **Frontend hosting:** AWS Amplify Hosting or S3 + CloudFront for the built React app.
- **Object storage (future photo-proof feature):** Amazon S3.
- **Secrets:** AWS Secrets Manager.
- **Monitoring:** CloudWatch (logs + basic metrics), free tier sufficient at portfolio scale.

Better Production-Style Deployment

- **Application hosting:** ECS Fargate (or EKS) running multiple API task replicas behind an Application Load Balancer with WebSocket-aware sticky-less routing (enabled by the Redis pub/sub design — no session affinity required).
- **Database hosting:** RDS PostgreSQL Multi-AZ with a read replica for analytics queries.
- **Cache:** ElastiCache Redis in cluster mode for higher availability.
- **CDN:** CloudFront in front of both the static frontend and (optionally) the API.
- **Monitoring:** CloudWatch + X-Ray distributed tracing, alerting via SNS to Slack/email.
- **Message broker upgrade:** Amazon SQS or MSK (managed Kafka) replacing the outbox table for webhook delivery at real production volume.

23. GitHub Repository

```
shiftpilot/
├── backend/
│   ├── src/main/java/com/shiftpilot/
│   │   ├── api/                # Controllers, WebSocket handlers
│   │   ├── domain/              # Task, Agent, Assignment, state machine
│   │   ├── service/             # TaskService, AutoAssignService, SlaMonitorService
│   │   ├── repository/          # Spring Data JPA repositories
│   │   ├── realtime/            # Redis pub/sub broadcast layer
│   │   ├── integration/         # Webhook dispatch, SMS stub
│   │   └── config/               # Security, WebSocket, Redis config
│   ├── src/main/resources/
│   │   ├── application.yml
│   │   └── db/migration/         # Flyway migrations
│   └── src/test/java/...
├── frontend/
│   ├── src/
│   └── package.json
├── tests/
│   └── e2e/                      # Playwright specs
├── docs/
│   ├── architecture.md
│   ├── architecture-diagram.png
│   └── state-machine-diagram.png
├── infra/
│   ├── docker/
│   ├── docker-compose.yml
│   └── aws/                       # Terraform/CDK templates (optional)
├── .github/workflows/
│   ├── ci.yml
│   └── deploy.yml
├── .env.example
├── .gitignore
├── LICENSE
└── README.md
```

24. README

Exact section outline: 1. **ShiftPilot** — tagline + build/license badges 2. **Live Demo** — public URL, demo dispatcher login, demo agent login (two-persona demo) 3. **The Problem** — field-ops coordination via WhatsApp/spreadsheets, referencing the Repzo market-validation point 4. **Screenshots** — dispatch board, agent mobile view, analytics 5. **Architecture** — diagram from §10 + a short note on the Redis pub/sub horizontal-scaling design 6. **Tech Stack** — table with justifications 7. **Features** 8. **Getting Started** — `docker compose up` 9. **Environment Setup** 10. **API Documentation** — Swagger link + WebSocket subscription example 11. **Database Schema** — ERD link 12. **Testing** — highlight the two-browser-session real-time e2e test specifically 13. **Security** 14. **Deployment** 15. **Roadmap** 16. **License**

---

25. Demo Experience

Intended recruiter journey: 1. Recruiter opens GitHub → README's first screenshot is the live Kanban board mid-action, immediately communicating "real-time ops tool." 2. Clicks **Live Demo** → dispatcher board, seeded with a realistic mix of tasks across all five status columns. 3. The README explicitly invites: "*Open this link in a second browser tab as the Agent Demo, and watch the dispatcher board update live as you tap through an agent's task.*" — this two-tab trick is the single most convincing thing a recruiter can experience, because it makes the real-time architecture undeniable rather than a claim in a README. 4. Recruiter (in the agent tab) taps through Accept → En Route → Arrived → Complete, watching the dispatcher tab's card visibly move columns in real time with zero refresh. 5. Recruiter checks the Analytics tab, sees populated charts from seeded historical data. 6. Returns to GitHub, watches the 2-minute video specifically narrating the Redis pub/sub architecture and the optimistic-locking race-condition fix.

Demo account details to provide: A "Dispatcher Demo" login and an "Agent Demo" login, both one-click from the live login screen, explicitly labeled for the two-tab experience described above.

26. LinkedIn Presentation

Project title: ShiftPilot — Real-Time Dispatch & Field Operations Platform

Short LinkedIn description: "Built ShiftPilot, a real-time field-dispatch platform (Spring Boot + Redis pub/sub + WebSockets) that replaces WhatsApp-and-spreadsheet coordination for delivery, home-service, and field-sales teams."

Full LinkedIn description: "Most SME field operations in Jordan are coordinated through phone calls and WhatsApp groups — no live visibility, no data, no way to catch a missed SLA before the customer complains.

I built ShiftPilot to fix that: a Spring Boot + PostgreSQL + Redis backend with a real-time Kanban dispatch board (WebSocket/STOMP, backed by Redis pub/sub so the API scales horizontally without sticky sessions), a rule-based auto-assignment engine, optimistic-locking concurrency control to prevent double-booking, and a scheduled SLA-risk monitor. Agents get a lightweight mobile task view; managers get performance analytics.

Try the live two-persona demo — open the dispatcher board and the agent view side by side and watch tasks move in real time. Docker + AWS + full test suite including a two-browser-session e2e test proving the real-time sync actually works.

Demo + source below."

Suggested launch post: "I got tired of seeing delivery and home-service businesses run their entire operation through WhatsApp — so I built ShiftPilot: a real-time dispatch board with live agent tracking, auto-assignment, and SLA alerts. The fun part was the Redis pub/sub architecture that lets the WebSocket layer scale across multiple servers with zero sticky sessions. Try the live demo — open two tabs and watch a task move in real time 👀 Link below."

27. CV Presentation

One-line project description: Built a real-time field-dispatch platform (Spring Boot, Redis pub/sub, WebSockets) with rule-based auto-assignment and SLA monitoring.

CV bullets: - Designed and built a real-time dispatch platform (Spring Boot, PostgreSQL, Redis) with a WebSocket/STOMP live board, enabling horizontal API scaling via Redis pub/sub fan-out instead of sticky-session WebSocket routing. - Implemented a task state-machine with optimistic-locking concurrency control, eliminating a double-assignment race condition under concurrent dispatcher actions. - Built a rule-based auto-assignment engine scoring agents by load and availability, plus a scheduled SLA-risk monitor flagging at-risk tasks before breach. - Shipped a two-persona (dispatcher + field-agent) experience with role-scoped authorization, HMAC-signed webhooks, and a Playwright end-to-end test proving real-time sync across concurrent sessions; deployed on AWS (ECS/RDS/ElastiCache) with Docker and GitHub Actions CI/CD.

Skills demonstrated: Java, Spring Boot, Spring Security, Spring Data JPA, WebSockets/STOMP, Redis (pub/sub + caching), PostgreSQL, optimistic concurrency control, REST API design, React, TypeScript, Docker, AWS, CI/CD, automated testing (JUnit, Testcontainers, Playwright, k6).

Technologies: Java 17, Spring Boot 3, Spring WebSocket, Spring Data JPA, PostgreSQL, Redis, React, TypeScript, TanStack Query, Docker, Docker Compose, GitHub Actions, AWS (ECS/RDS/ElastiCache), JUnit 5, Testcontainers, Playwright, k6.

28. Interview Preparation

Architecture questions: - Why Redis pub/sub for real-time broadcast instead of just keeping WebSocket sessions in-process? (In-process session maps break the moment you run more than one server instance — a client connected to server A never hears about an event handled by server B. Redis pub/sub decouples "who received the event" from "which server handled the request that caused it.") - Why is webhook delivery on a different reliability mechanism (Postgres outbox) than real-time broadcast (Redis pub/sub)? (Different consistency requirements — a missed live-board update self-heals on the next event or a refresh; a missed webhook is a silent, permanent failure to notify an external system unless explicitly guaranteed and retried.)

Database questions: - How does the schema prevent two dispatchers from assigning the same task simultaneously? (The `version` optimistic-locking column on `Task`, plus a unique partial index ensuring only one *active* assignment can exist per task at the database level — belt and suspenders, not relying on application logic alone.) - Why track `current_load` as a denormalized counter on `Agent` rather than computing it live with a `COUNT()` query? (Read-performance trade-off for a value read on every auto-assign decision; kept correct via a transactional update alongside every assignment/completion, an explicit denormalization trade-off worth being able to defend.)

API questions: - Why does `POST /tasks/{id}/assign` return 409 on a conflict instead of silently succeeding for the "losing" request? (Silent success would hide the double-booking bug entirely; an explicit 409 forces the client to refetch and show the dispatcher accurate state.)

Security questions: - How is it guaranteed that Agent A cannot fetch Agent B's tasks even with a valid, unexpired JWT? (Explicit ownership check in the service layer comparing `assignment.agentId` to the authenticated principal's agent ID — role-based access alone (`ROLE_AGENT`) is insufficient, since both agents share the same role; this is an object-level authorization concern, not just role-based access control.) - Why is the WebSocket subscription itself authorized server-side rather than trusting that "if they have a valid JWT, any subscription is fine"? (A valid JWT proves identity, not authorization to a specific topic — subscribing to another company's board topic must be explicitly rejected even for an authenticated user.)

Design decisions to understand deeply: the optimistic-locking choice over pessimistic row-locking (better throughput for a system with many concurrent reads and comparatively rare write conflicts); the decision to keep the auto-assign engine rule-based and explainable rather than ML-based (§16) even though "AI-powered dispatch" sounds more impressive — and why that's the *right* call for a safety/operationally-relevant decision.

Trade-offs to be ready to explain: BCrypt vs. Argon2id (contrast with the ClearVoice document — BCrypt is a defensible, well-supported Java-ecosystem default, Argon2id is marginally stronger; both are legitimate choices, and being able to articulate *why* a given project made a given choice is the actual interview signal, not memorizing "the one correct algorithm").

---

29. Development Roadmap

Phase 1 — Project Setup: Spring Boot + React scaffolding, Docker Compose skeleton with Postgres + Redis, CI skeleton. *Definition of done:* `docker compose up` boots an API returning 200 on `/actuator/health`.

Phase 2 — Database: Entities (§11), Flyway migrations, seed script (demo company, agents, sample tasks across all statuses). *Definition of done:* seed script populates a realistic demo dataset.

Phase 3 — Authentication: Manager/dispatcher email login, agent phone+PIN login, JWT issuance, role-based `@PreAuthorize` policies. *Definition of done:* both persona login flows work; an integration test confirms agent-task-ownership isolation.

Phase 4 — Core Backend (Task State Machine + Manual Assignment): Task CRUD, the `Task.transitionTo()` state machine, manual assignment with optimistic locking. *Definition of done:* an integration test proves two concurrent assignment requests for the same task result in exactly one success and one 409.

Phase 5 — Real-Time Layer (WebSocket + Redis Pub/Sub): STOMP endpoint, Redis pub/sub broadcast on status change, frontend `useTaskSocket()` hook. *Definition of done:* the two-browser-tab live-update experience works end-to-end locally.

Phase 6 — Frontend (Dispatch Board + Agent View): Kanban board UI, task slide-over, agent mobile task list. *Definition of done:* a dispatcher can create/assign a task and an agent can advance it through the full status pipeline via the UI.

Phase 7 — Automation (Auto-Assign + SLA Monitor + Webhooks): Scoring engine, scheduled SLA scan, webhook outbox + delivery worker. *Definition of done:* auto-assign correctly picks the least-loaded on-shift agent in a test scenario; an artificially-shortened SLA window correctly triggers an at-risk flag within the test run.

Phase 8 — Testing: Full unit/integration/e2e suite, specifically the flagship two-session Playwright test. *Definition of done:* CI green, including the concurrency and authorization-isolation tests.

Phase 9 — Docker: Finalize Dockerfiles and Compose setup with seed data for one-command demo bootstrap.

Phase 10 — CI/CD: GitHub Actions `ci.yml`/`deploy.yml`.

Phase 11 — Cloud Deployment: Provision AWS resources (App Runner/ECS, RDS, ElastiCache, Amplify/S3+CloudFront), wire secrets via Secrets Manager.

Phase 12 — Polish/Demo: Seed a rich two-persona demo dataset, record the architecture-focused walkthrough video emphasizing the real-time and concurrency-control design, finalize README with the "open two tabs" demo instructions.

---

30. Final Product Blueprint

```
PRODUCT: ShiftPilot — a real-time dispatch and field-operations platform
         for delivery, home-service, and field-sales teams.

PROBLEM: SME field operations in Jordan are coordinated manually via
         phone/WhatsApp, with no live visibility, no audit trail, and no
         performance data — a gap validated by the existence of
         Jordanian field-sales SaaS company Repzo.

USERS:   Dispatchers (create/assign tasks), field agents (mobile
         status-update view), operations managers (analytics + roster +
         settings), and indirectly, end customers (notification only).

CORE WORKFLOW: Dispatcher creates a task → auto-assign engine scores and
         picks an on-shift agent (or leaves it for manual assignment) →
         agent accepts and advances through en-route/arrived/completed →
         every transition broadcasts live via Redis pub/sub + WebSocket
         to the dispatcher board and triggers an outbox-queued webhook →
         a scheduled job flags tasks at risk of breaching their SLA
         window before they actually do.

FRONTEND: React 18 + TypeScript, one codebase serving both a Kanban
          dispatcher console and a mobile-optimized agent task list;
          TanStack Query with WebSocket-driven cache patching.

BACKEND:  Java 17 / Spring Boot 3, layered architecture with a
          self-enforcing Task state machine, Spring Data JPA +
          PostgreSQL, Redis for pub/sub broadcast and caching.

DATABASE: PostgreSQL — companies, users, agents, tasks, assignments,
          status_events, webhook_endpoints, webhook_deliveries,
          outbox_messages.

AI:       None in the core product by design (dispatch correctness is
          safety/operationally relevant and kept deterministic and
          explainable); an optional future statistical demand-forecast
          suggestion is human-reviewed only, never auto-applied.

INTEGRATIONS: Company webhooks, SMS provider (stub), transactional
          email, maps/geocoding (v2).

SECURITY: BCrypt password/PIN hashing, JWT + role-based + object-level
          authorization (agent-own-task enforcement), authorized
          WebSocket subscriptions, optimistic-locking concurrency
          control, HMAC-signed webhooks, TLS everywhere.

DEVOPS:   Trunk-based Git flow, GitHub Actions CI/CD, multi-stage Docker
          builds, Docker Compose for local dev with seeded two-persona
          demo data.

CLOUD:    AWS — App Runner/ECS, RDS for PostgreSQL, ElastiCache for
          Redis, Amplify/S3+CloudFront, Secrets Manager, CloudWatch.

MVP:      Task CRUD, manual assignment, state machine, real-time board,
          basic analytics, auth/RBAC.

FINAL VERSION: Rule-based auto-assignment, SLA-risk alerts, webhooks,
          live map view, route optimization, offline-first agent PWA,
          customer-facing tracking link, predictive staffing
          suggestions.
```